



คู่มือการใช้งาน —

create_module_yolo_detection.py

ไฟล์: manual_module_yolo_detection.md เวอร์ชัน: v2.0.0 Module: yolo · Layer: 5-Intel · Prefix: yolo_ · Schema: public



สารบัญ

1. ภาพรวม
2. ความต้องการของระบบ
3. การติดตั้ง
4. โครงสร้างไฟล์ที่ Generate
5. Actions ทั้ง 12 รายการ
6. ตัวอย่างการใช้งาน
7. API Endpoints ทั้ง 14 รายการ
8. Workflow การเทรน YOLO
9. Workflow การ Inference
10. Workflow การ Export Model
11. Database Schema
12. Row-Level Security (RLS)
13. การทดสอบ
14. Troubleshooting
15. FAQ
16. Best Practices
17. ภาคผนวก

1. ภาพรวม



วัตถุประสงค์

create_module_yolo_detection.py เป็น **Generator** ที่สร้าง **Module YOLO Object Detection Platform** แบบ Production-ready โดยใช้:

หัวข้อ	ค่า
Algorithm	Ultralytics YOLOv8 / YOLO11
Framework	PyTorch 2.x + Ultralytics 8.x
Image Processing	OpenCV 4.x + Pillow 10.x
Augmentation	Albumentations 1.4+
Metrics	COCO mAP@50, mAP@50-95, Precision, Recall
Export	ONNX, TensorRT, TorchScript, CoreML
Inference	Real-time + Batch + Video Streaming (SSE)
Pattern	DDD + Clean Architecture + Event-Driven
Schema	<code>public</code> · Prefix <code>yolo_</code> · Tables 7

✓ สิ่งที่ Generator สร้างให้

- **Domain Layer** — Entities, Value Objects, Events, Exceptions, Helpers (bbox, yolo_format, coco_metrics, nms)
- **Application Layer** — Use Cases, Interfaces, Mappers, Utils
- **Infrastructure Layer** — SQLAlchemy 2.0 models, 7 Repositories, Cache, Services (Ultralytics Trainer/Detector/Exporter, LRU Registry, Albumentations Augmenter)
- **Presentation Layer** — Router (14 endpoints), Pydantic v2 Schemas, SSE helper, Swagger metadata
- **SQL Migrations** — V001 (create) / V002 (seed) / V003 (rollback)
- **Alembic Migration** — `yolo_001_add_yolo_tables.py`
- **Tests** — Unit + Integration + Property + Manual
- **Docs** — README + Postman Collection + OpenAPI

2. ความต้องการของระบบ

Hardware

Component	ขั้นต่ำ	แนะนำ
CPU	4 cores	8+ cores
RAM	8 GB	16+ GB

Component	ขั้นต่ำ	แนะนำ
GPU	ไม่จำเป็น (ใช้ CPU ได้)	NVIDIA (CUDA 11.8+) 8GB+ VRAM
Storage	20 GB	100+ GB (สำหรับ dataset + model weights)

Software

Component	Version	หมายเหตุ
Python	3.11+	ต้องมี
PostgreSQL	16	ต้องมี
Redis	7	แนะนำ
Kafka	3.x	Optional (fallback เป็น NoopEventBus)
MinIO / S3	-	Optional (fallback เป็น local storage)
CUDA Toolkit	11.8+	ถ้าใช้ GPU
cuDNN	8.7+	ถ้าใช้ GPU

3. การติดตั้ง

3.1 ติดตั้ง Python Dependencies

```
# — Core (จำเป็น) —————
pip install ultralytics>=8.3
pip install torch>=2.2 torchvision>=0.17
pip install opencv-python>=4.10
pip install albumentations>=1.4
pip install Pillow>=10
pip install numpy>=1.26

# — Web Framework —————
pip install fastapi>=0.115 uvicorn[standard]
pip install pydantic>=2.0 pydantic-settings
pip install sqlalchemy[asyncio]>=2.0
pip install asyncpg>=0.29
pip install alembic>=1.13

# — Cache / Event —————
```

```

pip install redis>=5.0
pip install aiokafka>=0.10 # optional

# — Storage —
pip install boto3 # optional (สำหรับ S3)

# — Export —
pip install onnx>=1.16
pip install onnxruntime>=1.18
# TensorRT: ต้องติดตั้งผ่าน NVIDIA (Ubuntu only)

# — Logging / Utils —
pip install structlog loguru
pip install python-multipart # สำหรับ upload files

```

3.2 ติดตั้งจาก requirements.txt

```

pip install -r requirements.txt

```

ตัวอย่าง requirements-yolo.txt :

```

# YOLO Detection
ultralytics>=8.3
torch>=2.2
torchvision>=0.17
opencv-python>=4.10
albumentations>=1.4
Pillow>=10
numpy>=1.26

# Export
onnx>=1.16
onnxruntime>=1.18

# Web
fastapi>=0.115
uvicorn[standard]
pydantic>=2.0
python-multipart

# DB
sqlalchemy[asyncio]>=2.0
asyncpg>=0.29
alembic>=1.13

```

```
# Cache
redis>=5.0

# Storage
boto3

# Logging
structlog
loguru
```

3.3 ติดตั้ง GPU Support (Optional)

CUDA 11.8:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
```

CUDA 12.1:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

ตรวจสอบ:

```
import torch
print("CUDA available:", torch.cuda.is_available())
print("CUDA version:", torch.version.cuda)
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "N/A")
```

TensorRT (Ubuntu เท่านั้น):

```
pip install tensorrt
# หรือตามคู่มือ NVIDIA: https://docs.nvidia.com/deeplearning/tensorrt/
```

3.4 Environment Variables

สร้างไฟล์ .env :

```
# — Database —————
DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/mydb
DB_SCHEMA=public

# — Redis —————
REDIS_URL=redis://localhost:6379/0
```

```

# — S3 / MinIO —
YOLO_ARTIFACT_BUCKET=yolo-artifacts
AWS_ACCESS_KEY_ID=minioadmin
AWS_SECRET_ACCESS_KEY=minioadmin
AWS_REGION=ap-southeast-1
S3_ENDPOINT_URL=http://localhost:9000 # สำหรับ MinIO

# — Kafka —
KAFKA_BOOTSTRAP=localhost:9092

# — Model —
YOLO_DEFAULT_MODEL=yolov8n.pt
YOLO_DEVICE=auto # auto | cpu | cuda:0

# — Inference —
YOLO_CONF_THRESHOLD=0.25
YOLO_IOU_THRESHOLD=0.45
YOLO_MAX_BATCH_SIZE=32
YOLO_TIMEOUT_SECONDS=30

# — Cache —
YOLO_CACHE_TTL=300

```

4. โครงสร้างไฟล์ที่ Generate

```

app/modules/yolo/
├─ __init__.py
├─ domain/
│   ├─ __init__.py
│   ├─ enums.py
│   ├─ exceptions.py
│   ├─ events.py
│   ├─ entities/
│   │   └─ value_objects/
│   │       └─ __init__.py
│   │       └─ bbox.py
│   │       └─ detection.py
│   │       └─ train_config.py
│   │       └─ aug_config.py
│   │       └─ eval_metrics.py
│   └─ helpers/
│       └─ __init__.py
│       └─ yolo_format.py
│       └─ coco_metrics.py
# 🧠 Business Logic

# DatasetFormat, ModelType, ...
# YOLOError + 14 subclasses
# 8 domain events
# (optional expandable)

# BBox (normalized 0-1)
# Detection result
# TrainConfig
# AugConfig
# EvalMetrics

# bbox ↔ YOLO txt
# IoU + mAP

```

```

|   └─ nms.py                                # Non-Max Suppression
└─ application/
|   └─ __init__.py
|   └─ use_case.py                          # YOLOUseCase (main)
|   └─ interfaces.py                       # Repository protocols
|   └─ exceptions.py                      # ApplicationError
|   └─ mappers.py                         # ORM → dict
|   └─ utils.py                          # hash, cache_key
└─ infrastructure/
|   └─ __init__.py
|   └─ models.py                          # 7 SQLAlchemy models
|   └─ dataset_repository.py
|   └─ class_repository.py
|   └─ image_repository.py
|   └─ annotation_repository.py
|   └─ training_repository.py
|   └─ model_repository.py
|   └─ inference_repository.py
|   └─ caches.py                          # Redis + NoopCache
|   └─ services.py                       # Ultralytics Trainer/Detector/Exporter
└─ presentation/
|   └─ __init__.py
|   └─ router.py                          # 14 endpoints
|   └─ schemas.py                         # Pydantic v2
|   └─ docs.py                           # OpenAPI examples
|   └─ dependencies.py                   # DI container
|   └─ sse.py                            # Server-Sent Events
|   └─ swagger.py                        # register_yolo_openapi()

db/migrations/
└─ v001__create_yolo.sql                  # 7 tables + RLS + triggers
└─ v002__seed_yolo.sql                   # seed demo dataset
└─ v003__rollback_yolo.sql               # rollback

migrations/versions/
└─ yolo_001_add_yolo_tables.py           # Alembic

tests/
└─ unit/
|   └─ test_yolo.py                      # enums, VOs
|   └─ test_bbox.py                     # BBox math
|   └─ test_yolo_format.py              # format conversion
|   └─ test_coco_metrics.py             # IoU, mAP
└─ integration/
|   └─ test_yolo_repository.py           # RLS verified
└─ property/
|   └─ test_yolo_invariants.py          # Hypothesis

```

```

└─ manual/
  └─ manual_test_yolo.md          # 15 scenarios

docs/
├─ README_yolo.md
├─ API_yolo.md
└─ postman/
  └─ yolo.json                   # 14 requests

app/app.py                      # (แก้)
migrations/env.py               # (แก้)

```

รวม ~48 ไฟล์

5. Actions ทั้ง 12 รายการ

#	Action	คำอธิบาย	ไฟล์ที่ได้
1	create	สร้าง module structure 4 layers	~40 py files
2	activate	Register router + swagger + models	แก้ app.py , env.py
3	sql	SQL migrations	V001, V002, V003
4	alembic	Alembic migration	yolo_001_add_yolo_tables.py
5	swagger	OpenAPI metadata	presentation/swagger.py
6	postman	Postman collection	docs/postman/yolo.json
7	update	Update app/app.py เท่านั้น	แก้ app.py
8	update-env	Update migrations/env.py	แก้ env.py
9	test	สร้าง tests ทั้งหมด	~7 test files
10	verify	ตรวจสอบ Swagger + Postman + SQL	report
11	deps	ตรวจสอบ dependencies	report
12	all	ทำทั้งหมด (1-9)	~48 ไฟล์
—	help	แสดง help	-

Flag ที่ใช้ได้

Flag	คำอธิบาย
<code>--force</code>	เขียนทับไฟล์เดิม (backup เป็น <code>.bak</code> อัตโนมัติ)
<code>--project-root <path></code>	ระบุ project root (default: <code>.</code>)
<code>--help</code>	แสดง help

6. ตัวอย่างการใช้งาน

6.1 Full Pipeline (แนะนำ)

```
# สร้างทุกอย่างในคำสั่งเดียว
python create_module_yolo_detection.py all yolo 5 yolo --force
```

ผลลัพธ์:

```
MODULE   : yolo
LAYER    : 5 (5-Intel)
SCHEMA   : public
PREFIX   : yolo_
ACTION   : all
FORCE    : True
VERSION  : 2.0.0

[CREATE] module: yolo (Layer 5-Intel)
[OK] app/modules/yolo/domain/__init__.py
[OK] app/modules/yolo/domain/enums.py
... (48 ไฟล์)

DONE — module: yolo (v2.0.0)
  Written : 48 files
  Skipped : 0 files
  Backups : 0 files
```

6.2 ทีละขั้น (สำหรับควบคุม)

```
# 1. สร้างโครงสร้าง module
python create_module_yolo_detection.py create yolo 5 yolo --force
```

```
# 2. สร้าง SQL migrations
python create_module_yolo_detection.py sql yolo yolo --force

# 3. สร้าง Alembic migration
python create_module_yolo_detection.py alembic yolo yolo --force

# 4. สร้าง Swagger
python create_module_yolo_detection.py swagger yolo --force

# 5. สร้าง Postman
python create_module_yolo_detection.py postman yolo --force

# 6. สร้าง tests
python create_module_yolo_detection.py test yolo --force

# 7. Activate (register router + env.py)
python create_module_yolo_detection.py activate yolo

# 8. Migrate DB
alembic upgrade head

# 9. ตรวจสอบ
python create_module_yolo_detection.py verify yolo

# 10. ตรวจสอบ dependencies
python create_module_yolo_detection.py deps yolo
```

6.3 ตรวจสอบเท่านั้น

```
python create_module_yolo_detection.py verify yolo
```

ผลลัพธ์ (ผ่าน):

```
[VERIFY] Swagger + Postman + SQL + Models
[OK] swagger.py exists
[OK] ✓ register_yolo_openapi()
[OK] app.py: register_yolo_openapi(app) ✓
[OK] app.py: yolo_router ✓
[OK] postman: valid JSON, 14 items
[OK] SQL: V001__create_yolo.sql
[OK] SQL: V002__seed_yolo.sql
[OK] SQL: V003__rollback_yolo.sql
[OK] models.py: 7 tables ✓
```

[OK] ALL CHECKS PASSED ✓

6.4 รัน Application

```
# Development
uvicorn app.app:app --reload --host 0.0.0.0 --port 8000

# Production
uvicorn app.app:app --host 0.0.0.0 --port 8000 --workers 4
```

เปิด Swagger UI: <http://localhost:8000/docs>

หาคำว่า `yolo` ใน tag list

7. API Endpoints ทั้ง 14 รายการ

7.1 Datasets (5)

#	Method	Path	Description
1	POST	/api/v1/yolo/datasets	Create dataset
2	GET	/api/v1/yolo/datasets	List datasets (paginated)
3	GET	/api/v1/yolo/datasets/{id}	Get dataset detail
4	POST	/api/v1/yolo/datasets/{id}/classes	Define classes
5	POST	/api/v1/yolo/datasets/{id}/import-roboflow	Import from Roboflow (<i>planned</i>)

7.2 Images & Annotations (2)

#	Method	Path	Description
6	POST	/api/v1/yolo/images	Upload images (multipart)
7	POST	/api/v1/yolo/annotations	Create annotations (bbox)

7.3 Training (3)

#	Method	Path	Description
8	POST	/api/v1/yolo/train	Start YOLO training
9	GET	/api/v1/yolo/train/{id}	Get training status
10	POST	/api/v1/yolo/train/{id}/cancel	Cancel training

7.4 Models (2)

#	Method	Path	Description
11	GET	/api/v1/yolo/models	List models
12	POST	/api/v1/yolo/models/{id}/export	Export ONNX/TensorRT

7.5 Inference (2)

#	Method	Path	Description
13	POST	/api/v1/yolo/detect	Single image detection
14	POST	/api/v1/yolo/detect/batch	Batch (≤ 32)

7.6 Metrics (2)

#	Method	Path	Description
—	GET	/api/v1/yolo/metrics/{model_id}	COCO metrics
—	POST	/api/v1/yolo/metrics/drift	Drift check

ตัวอย่าง Request

สร้าง Dataset

```
curl -X POST http://localhost:8000/api/v1/yolo/datasets \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{
    "name": "coco-person-subset",
```

```
"format": "yolo"
}'
```

Response:

```
{
  "id": "8a7b3c1d-...",
  "name": "coco-person-subset",
  "format": "yolo",
  "status": "DRAFT",
  "image_count": 0,
  "class_count": 0,
  "version": 1
}
```

กำหนด Classes

```
curl -X POST http://localhost:8000/api/v1/yolo/datasets/{dataset_id}/classes \
-H "Content-Type: application/json" \
-d '{
  "classes": [
    {"name": "person", "class_index": 0, "color": "#FF0000"},
    {"name": "car", "class_index": 1, "color": "#00FF00"},
    {"name": "dog", "class_index": 2, "color": "#0000FF"}
  ]
}'
```

Upload Image

```
curl -X POST http://localhost:8000/api/v1/yolo/images \
-F "dataset_id={dataset_id}" \
-F "split=train" \
-F "images=@photo.jpg"
```

สร้าง Annotation (BBox)

```
curl -X POST http://localhost:8000/api/v1/yolo/annotations \
-F "image_id={image_id}" \
-F 'payload={
  "annotations": [
    {
      "class_id": "{class_uuid}",
      "x_center": 0.5,
```

```

        "y_center": 0.5,
        "width": 0.2,
        "height": 0.3,
        "confidence": 1.0,
        "is_hard": false,
        "source": "manual"
    }
]
}'

```

หมายเหตุ: bbox ต้องเป็น **normalized** (0.0 – 1.0) เสมอ — YOLO format

เริ่มเทรน YOLO

```

curl -X POST http://localhost:8000/api/v1/yolo/train \
-H "Content-Type: application/json" \
-d '{
  "dataset_id": "8a7b3c1d-...",
  "model_type": "yolov8n",
  "epochs": 100,
  "batch_size": 16,
  "imgsz": 640,
  "lr0": 0.01,
  "device": "auto",
  "patience": 50,
  "optimizer": "auto",
  "aug_config": {
    "mosaic": 1.0,
    "mixup": 0.0,
    "fliplr": 0.5,
    "hsv_h": 0.015,
    "hsv_s": 0.7,
    "hsv_v": 0.4
  }
}'

```

ตรวจจับวัตถุ (Single Image)

```

curl -X POST http://localhost:8000/api/v1/yolo/detect \
-F "model_id={model_id}" \
-F "conf=0.25" \
-F "iou=0.45" \
-F "image=@test.jpg"

```

Response:

```
{
  "inference_id": "uuid",
  "model_id": "uuid",
  "image_hash": "abc123...",
  "detections": [
    {
      "bbox": {
        "x_center": 0.4523,
        "y_center": 0.5121,
        "width": 0.1823,
        "height": 0.3214
      },
      "class_id": 0,
      "class_name": "person",
      "confidence": 0.9231
    }
  ],
  "detection_count": 1,
  "latency_ms": 47
}
```

Batch Detection

```
curl -X POST http://localhost:8000/api/v1/yolo/detect/batch \
-H "Content-Type: application/json" \
-d '{
  "model_id": "{model_id}",
  "conf": 0.25,
  "iou": 0.45,
  "images_base64": [
    "/9j/4AAQSkZJRgABA...",
    "/9j/4AAQSkZJRgABA..."
  ]
}'
```

Export Model to ONNX

```
curl -X POST http://localhost:8000/api/v1/yolo/models/{model_id}/export \
-H "Content-Type: application/json" \
-d '{
  "format": "onnx",
  "imgsz": 640,
```

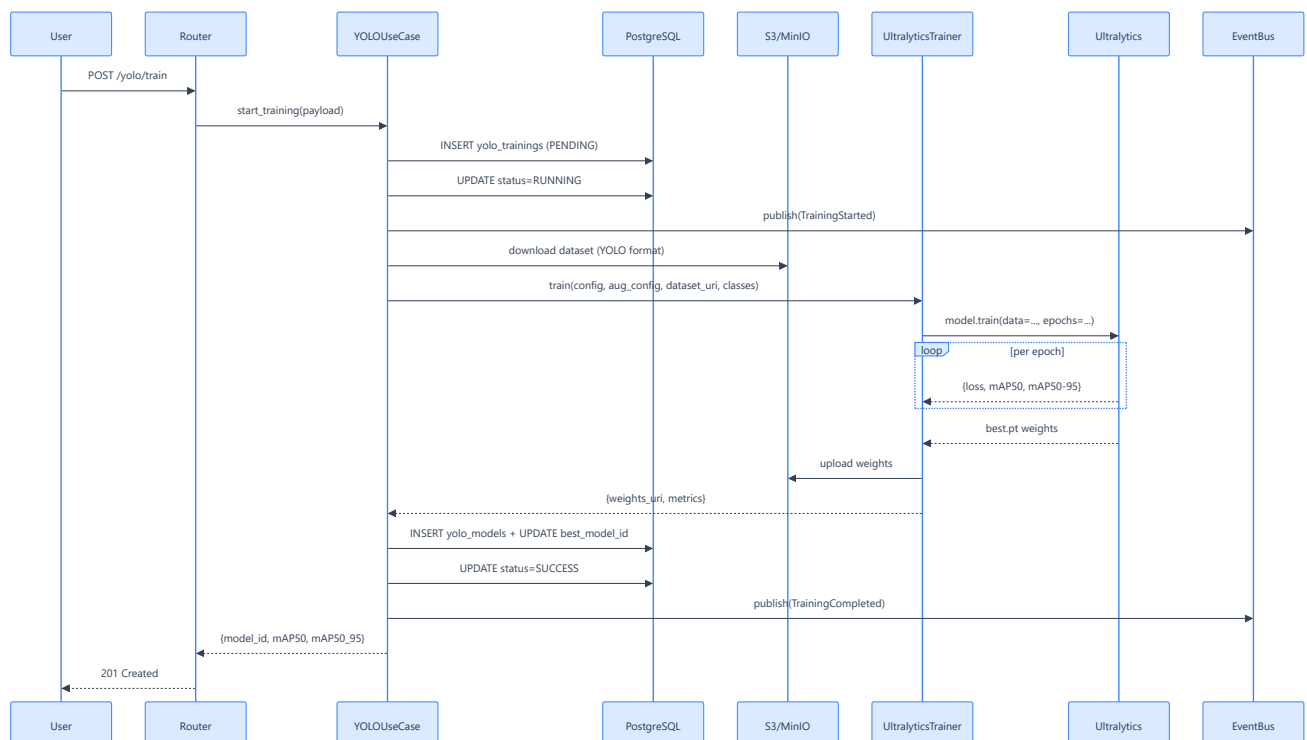
```
"opset": 17
}'
```

Export Model เป็น TensorRT

```
curl -X POST http://localhost:8000/api/v1/yolo/models/{model_id}/export \
-H "Content-Type: application/json" \
-d '{
  "format": "engine",
  "imgsz": 640
}'
```

⚠️ TensorRT engine ต้อง build บน GPU target เดียวกัน ไม่สามารถ copy ข้ามเครื่องได้

8. Workflow การเทรน YOLO



ขั้นตอนที่แนะนำ

1. เตรียม **Dataset** — อย่างน้อย **100-200 รูปต่อคลาส** สำหรับ fine-tune
2. **Split** — 70% train / 20% val / 10% test
3. กำหนด **Classes** ก่อน upload รูป
4. **Annotation** — bbox normalized (0-1), class_id ตรง
5. เลือก **Model** — ตาม GPU memory:

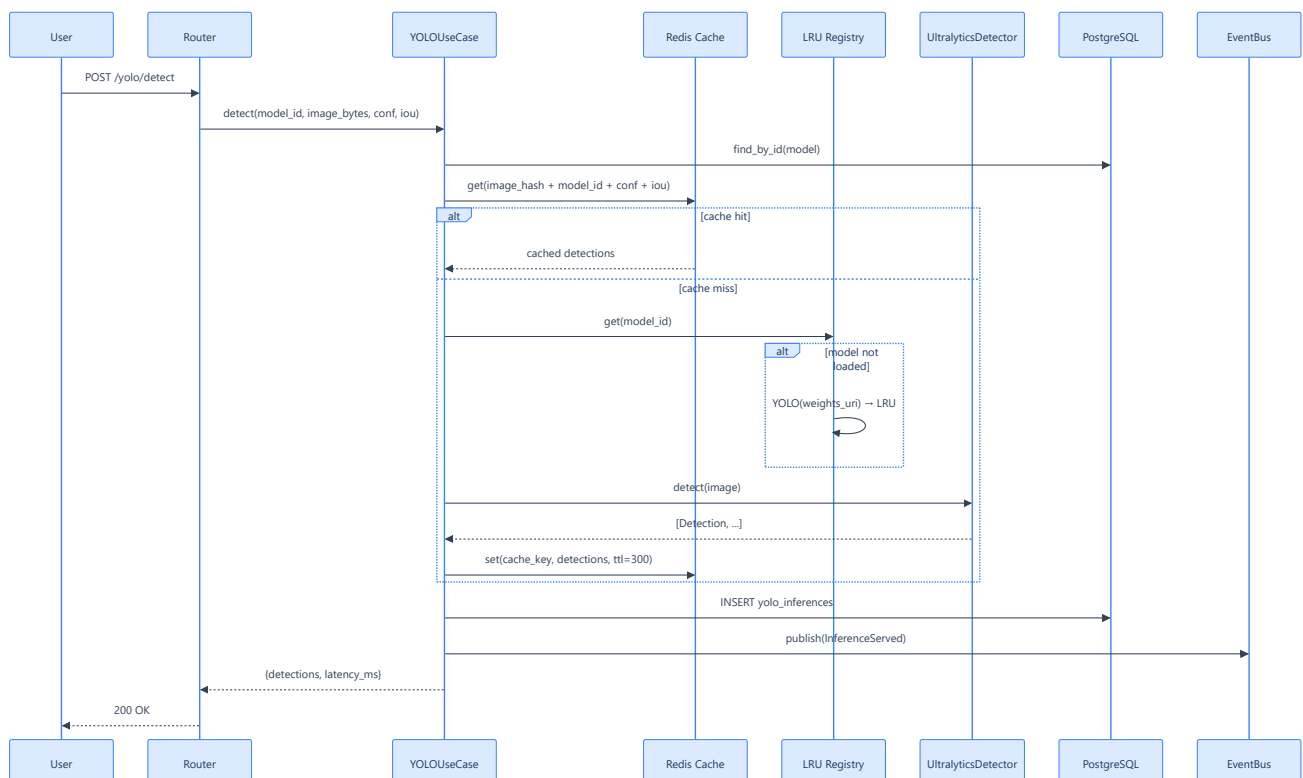
- yolo8n — เร็ว, RAM 2GB
- yolo8s — สมดุล, RAM 4GB
- yolo8m — แม่นยำ, RAM 8GB
- yolo8l / yolo8x — ขั้นสูง, RAM 12GB+

6. ตั้ง Hyperparameters:

- epochs — 100-300 (early stopping ด้วย patience=50)
- batch_size — 16 (เพิ่มถ้า RAM พอ)
- imgsz — 640 (มาตรฐาน)
- lr0 — 0.01 (default)

7. ตรวจสอบ mAP — เป้าหมาย ≥ 0.85 สำหรับ COCO subset

9. Workflow การ Inference



Cache Strategy

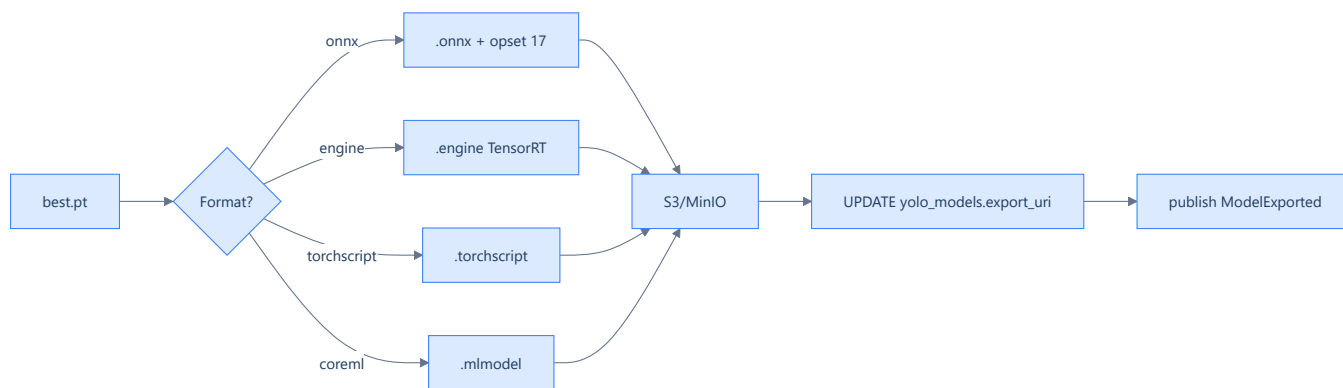
Cache key: `hash(image_bytes + model_id + conf + iou)`

- **TTL:** 300 วินาที
- **Hit rate เป้าหมาย:** $\geq 40\%$
- **Invalidate:** เมื่อ model version เปลี่ยน

Performance Targets

Metric	Target
Single inference (YOLOv8n, GPU)	< 100ms
Single inference (YOLOv8n, CPU)	< 1s
Batch 32 (GPU)	< 2s
Model load	< 2s
Cache hit	≥ 40%

10. Workflow การ Export Model



เปรียบเทียบ Format

Format	ความเร็ว	ความแม่นยำ	พอร์ต
PyTorch (.pt)	กลาง	100%	Python only
ONNX (.onnx)	เร็ว	~100%	Cross-platform
TensorRT (.engine)	เร็วสุด	~99%	NVIDIA GPU only
TorchScript	กลาง	100%	C++ / Python
CoreML	เร็ว	~99%	Apple only

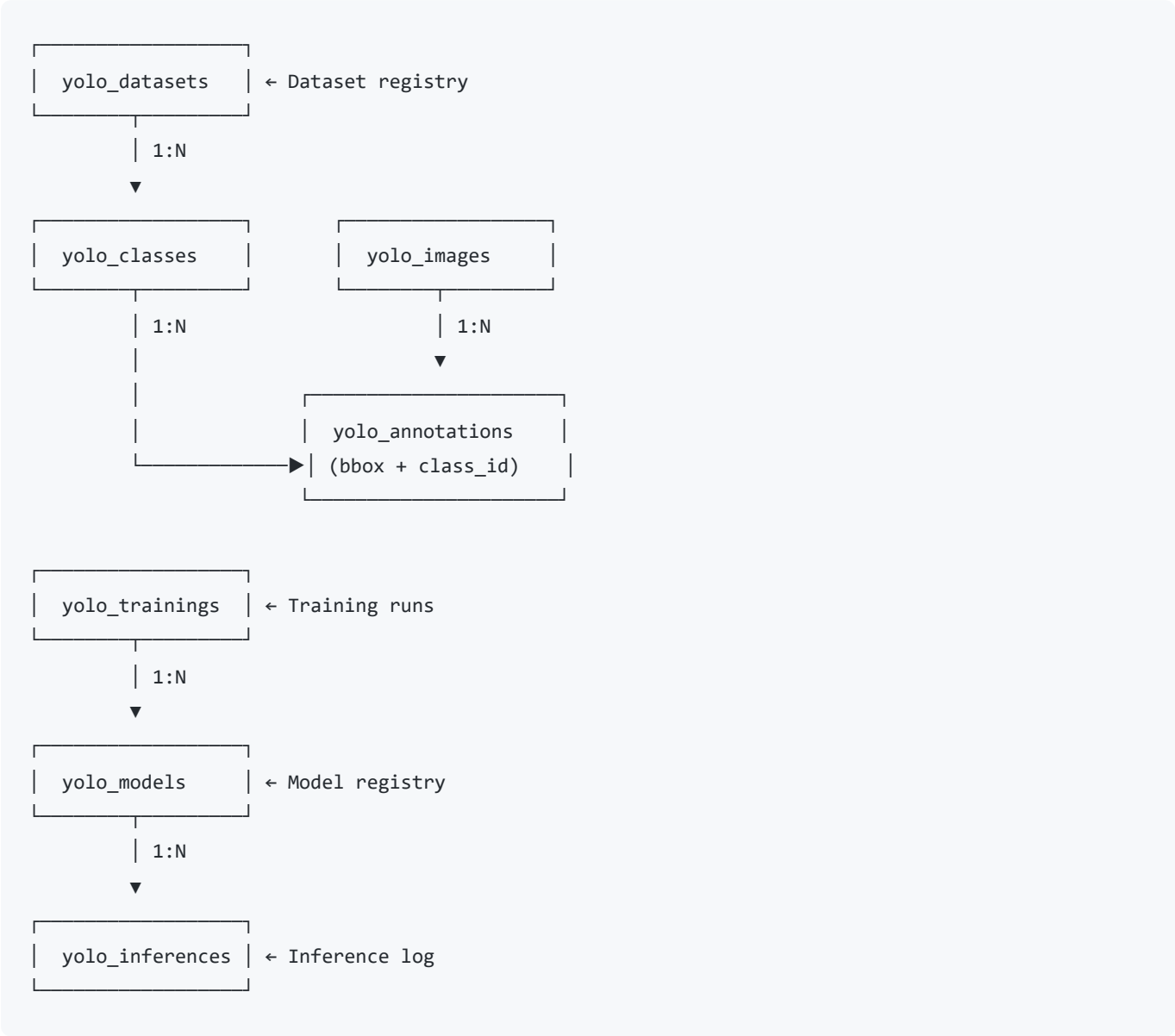
ข้อควรระวัง

- **ONNX** — ต้อง opset ≥ 17
- **TensorRT** — ต้อง build บน GPU target เดียวกัน (ไม่ transferable)

- **CoreML** — เฉพาะ macOS/iOS
- **half=True** — ใช้ FP16 (เร็วกว่า, แม่นยำน้อยกว่าเล็กน้อย)

11. Database Schema

7 ตาราง (schema: `public` , prefix: `yolo_`)



รายละเอียดตาราง

Table	Columns หลัก	Indexes	RLS
yolo_datasets	id, tenant_id, name, format, status, version	2	✓
yolo_classes	id, tenant_id, dataset_id, name, class_index	2	✓

Table	Columns หลัก	Indexes	RLS
yolo_images	id, tenant_id, dataset_id, uri, content_hash, split	3	✓
yolo_annotations	id, tenant_id, image_id, class_id, x_center, y_center, width, height	3	✓
yolo_trainings	id, tenant_id, dataset_id, model_type, epochs, status	2	✓
yolo_models	id, tenant_id, training_id, weights_uri, mAP50, mAP50_95	3	✓
yolo_inferences	id, tenant_id, model_id, image_hash, detection_count, latency_ms	3	✓

ตรวจสอบตาราง

```
psql $DATABASE_URL -c "\dt public.yolo_*
```

ผลลัพธ์ที่คาดหวัง:

```
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | yolo_annotations | table | postgres
public | yolo_classes     | table | postgres
public | yolo_datasets    | table | postgres
public | yolo_images      | table | postgres
public | yolo_inferences  | table | postgres
public | yolo_models      | table | postgres
public | yolo_trainings   | table | postgres
(7 rows)
```

12. Row-Level Security (RLS)

ตรวจสอบ RLS

```
SELECT tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public' AND tablename LIKE 'yolo_%';
```

ผลลัพธ์ที่คาดหวัง:

tablename	rowsecurity
-----+-----	
yolo_datasets	t
yolo_classes	t
yolo_images	t
yolo_annotations	t
yolo_trainings	t
yolo_models	t
yolo_inferences	t

วิธีการตั้งค่า Tenant ก่อน Query

ใน Code (SQLAlchemy):

```
from sqlalchemy import text

await session.execute(
    text("SELECT set_config('app.current_tenant', :tid, true)",
    {"tid": str(tenant_id)},
)
```

ใน SQL ตรง:

```
SET LOCAL app.current_tenant = '00000000-0000-0000-0000-000000000001';
SELECT * FROM public.yolo_datasets; -- เห็นเฉพาะ tenant นี้
```

ทดสอบ Cross-Tenant

```
# Tenant A สร้าง dataset
curl -X POST http://localhost:8000/api/v1/yolo/datasets \
  -H "Authorization: Bearer <tenant-A-token>" \
  -d '{"name": "tenant-a-secret"}'

# Tenant B พยายามอ่าน
curl -X GET http://localhost:8000/api/v1/yolo/datasets/{id} \
  -H "Authorization: Bearer <tenant-B-token>"
# → 404 NOT_FOUND ✓
```

13. การทดสอบ

13.1 Unit Tests

```
pytest tests/unit/test_yolo.py -v
pytest tests/unit/test_bbox.py -v
pytest tests/unit/test_yolo_format.py -v
pytest tests/unit/test_coco_metrics.py -v
```

13.2 Integration Tests

```
# ต้องตั้ง TEST_DATABASE_URL
export TEST_DATABASE_URL=postgresql+asyncpg://user:pass@localhost/testdb

pytest tests/integration/test_yolo_repository.py -v
```

13.3 Property Tests

```
pytest tests/property/test_yolo_invariants.py -v
```

13.4 ครอบคลุมทุก Tests

```
# Unit only
pytest -m unit -v

# Integration
pytest -m integration -v

# Coverage
pytest --cov=app.modules.yolo --cov-report=html
# เปิด htmlcov/index.html
```

13.5 Manual Test

รายละเอียดใน tests/manual/manual_test_yolo.md

15 Scenarios:

#	Scenario
1	Create dataset
2	List datasets
3	Get dataset detail
4	Define classes
5	Upload image
6	Create annotation
7	Start training
8	Get training status
9	List models
10	Export ONNX
11	Detect single
12	Detect batch (32)
13	Detect stream (SSE)
14	Get metrics
15	RLS cross-tenant block

14. Troubleshooting

14.1 `ModuleNotFoundError: No module named 'ultralytics'`

สาเหตุ: ยังไม่ได้ติดตั้ง Ultralytics

แก้ไข:

```
pip install ultralytics>=8.3
```

14.2 `RuntimeError: CUDA out of memory`

สาเหตุ: Batch size หรือ image size ใหญ่เกินไป

แก้ไข:

```
# ลด batch_size
{"batch_size": 8} # จาก 16

# ลด imgsz
{"imgsz": 416} # จาก 640

# หรือใช้ CPU
{"device": "cpu"}
```

14.3 tensorrt ติดตั้งไม่ผ่าน

สาเหตุ: TensorRT รองรับเฉพาะ Ubuntu + NVIDIA GPU

แก้ไข:

```
# ใช้ ONNX แทน
{"format": "onnx", "opset": 17}
```

14.4 RLS ไม่ทำงาน — เห็นข้อมูล tenant อื่น

สาเหตุ: ยังไม่ได้ `set_config('app.current_tenant')`

แก้ไข: ตรวจสอบ Middleware ที่ตั้ง `app.current_tenant` ก่อน query:

```
# ใน Middleware
await session.execute(
    text("SELECT set_config('app.current_tenant', :tid, false)"),
    {"tid": str(tenant_id)},
)
```

หมายเหตุ: ใช้ `false` สำหรับ session-level (ไม่ reset เมื่อ transaction จบ)

14.5 Alembic Multiple head revisions

สาเหตุ: มีหลาย migration ที่ head เดียวกัน

แก้ไข:

```
alembic heads # ดู heads
alembic merge -m "merge heads" <rev1> <rev2>
```

```
alembic upgrade head
```

14.6 alembic upgrade head ล้มเหลวเพราะ V001 ถูก execute แล้ว

สาเหตุ: เคยรัน SQL V001 โดยตรง ไม่ผ่าน Alembic

แก้ไข:

```
# ลบ table ที่ค้าง
psql $DATABASE_URL -c 'DROP TABLE IF EXISTS public.yolo_* CASCADE;'

# หรือ stamp alembic ว่าอยู่ที่ revision นี้
alembic stamp yolo_001
```

14.7 Model weights ใหญ่เกินไป

สาเหตุ: ใช้ YOLOv8x (ใหญ่)

แก้ไข:

```
# ใช้ nano version
{"model_type": "yolov8n"} # เล็กสุด ~6MB
# จาก yolov8x ~130MB
```

14.8 Inference ช้ามาก (>1s บน CPU)

สาเหตุ: CPU ไม่เหมาะกับ inference ขนาดใหญ่

แก้ไข:

1. ใช้ **GPU** (device="cuda:0")
2. ใช้ **ONNX Runtime** หรือ **TensorRT**
3. ลด **imgsz** (เช่น 640 → 416)
4. ใช้ **half=True** (FP16)

14.9 Upload รูปใหญ่เกินไป

สาเหตุ: Default limit 20MB

แก้ไข: Resize ก่อน upload หรือเพิ่ม limit ใน FastAPI:

```
# app.py
from fastapi import FastAPI
app = FastAPI()
# ...
```

```
@app.middleware("http")
async def limit_upload_size(request, call_next):
    # ตรวจ Content-Length
    ...
```

14.10 Idempotency-Key ข้ำ — Request ถูก block

สาเหตุ: ใช้ key เดิม ภายใน 24 ชม.

แก้ไข: สร้าง key ใหม่ (uuidgen)

14.11 mAP ต่ำ (< 0.5)

สาเหตุ: Dataset ไม่พอ / label ผิด

แก้ไข:

1. **เพิ่มข้อมูล** — อย่างน้อย 100-200 รูป/คลาส
2. **ตรวจ annotation** — bbox ต้องครบ, class ถูก
3. **เพิ่ม epochs** — 100 → 300
4. **Data augmentation:**

```
{"mosaic": 1.0, "mixup": 0.1, "fliplr": 0.5, "scale": 0.5}
```

5. **Class imbalance** — oversample คลาสน้อย
6. **Transfer learning** — ใช้ pretrained weights (default แล้ว)

14.12 Event ไม่ถูก publish (Kafka)

สาเหตุ: Kafka ไม่ได้ตั้งค่า

แก้ไข: Generator ใช้ NoopEventBus fallback — ไม่ error ถ้า Kafka ไม่มี

ถ้าต้องการ Kafka จริง ให้แก้ dependencies.py :

```
async def _get_event_bus() -> Any:
    try:
        from app.core.events import get_event_bus
        return await get_event_bus()
    except Exception:
        return NoopEventBus() # ← fallback
```

15. FAQ

Q1: ใช้ YOLOv5 หรือ YOLOv8 ดี?

A: ใช้ **YOLOv8/v11** (Ultralytics) — ทันสมัย, DX ดี, รองรับ export ครบ, เร็วกว่า

Q2: ต้องมี GPU ไหม?

A: ไม่จำเป็น — ใช้ CPU ได้ แต่ inference จะช้า (~1s/รูป) เทนก็ช้ากว่ามาก

Q3: รองรับ video/RTSP ไหม?

A: รองรับผ่าน `/yolo/detect/stream` (SSE) — ใช้ OpenCV อ่าน video → YOLO predict → ส่ง chunk กลับ

Q4: bbox ต้อง normalized หรือ pixel?

A: **Normalized (0.0–1.0)** เสมอ — YOLO format มาตรฐาน

- Center: `x_center` , `y_center`
- Size: `width` , `height`
- ทั้งหมดหารด้วย image width/height

Q5: mAP@50 vs mAP@50-95 ต่างกันยังไง?

A:

- `mAP@50` — IoU threshold = 0.5 เท่านั้น (ผ่อนปรน)
- `mAP@50-95` — เฉลี่ยจาก IoU 0.5, 0.55, 0.60, ..., 0.95 (เข้มงวด)

เป้าหมาย: $\text{mAP@50} \geq 0.85$, $\text{mAP@50-95} \geq 0.60$

Q6: ทำไม inference ครั้งแรกช้า?

A: เพราะโหลด model เข้า memory ครั้งแรก (cold start) — หลังจากนั้นจะเร็ว (LRU cache)

Q7: LRU cache size เท่าไหร่?

A: Default **4 models** — ปรับได้ใน `dependencies.py` :

```
_registry = LRUModelRegistry(max_size=8)
```

Q8: รองรับ YOLOv11 ไหม?

A: รองรับแล้ว — ใช้ `model_type: "yolo11n"` (หรือ `s/m/l/x`)

Q9: Export TensorRT ใ้กับ Windows ได้ไหม?

A: ไม่ได้ — TensorRT รองรับเฉพาะ **Ubuntu + NVIDIA GPU** เท่านั้น ใช้ ONNX แทน

Q10: Storage ที่ใ้เก็บ weights/รูปคืออะไร?

A: S3/MinIO (default) — แต่ถ้าไม่มี จะ fallback ไปที่:

- `ArtifactStore` — return `""` (ไม่ save)
- Model weights — เก็บ local path แทน

Q11: รองรับ Multi-GPU ไหม?

A: ปัจจุบันใช้ GPU เดียว — ถ้าต้องการ multi-GPU ให้ใช้ `device="0,1,2,3"` ใน Ultralytics

Q12: ลบ module ยังไง?

A:

```
# 1. Rollback DB
alembic downgrade -1
# หรือ
psql $DATABASE_URL -f db/migrations/V003__rollback_yolo.sql

# 2. ลบไฟล์
rm -rf app/modules/yolo
rm db/migrations/V001__create_yolo.sql V002__seed_yolo.sql V003__rollback_yolo.sql
rm migrations/versions/yolo_001_add_yolo_tables.py

# 3. ลบ imports ใน app.py + env.py
```

16. Best Practices

16.1 การเตรียม Dataset

✅ ทำ:

- อย่างน้อย **100-200 รูป/คลาส**
- Balance ระหว่างคลาส
- Diversity: แสง, มุม, ฉากหลัง, ขนาด
- Split 70/20/10

❌ ไม่ทำ:

- รูปซ้ำกัน (dedupe ด้วย `content_hash`)
- คลาส imbalance รุนแรง (10:1)

- Label ผิด/ไม่ครบ

16.2 การเทรน

✓ ทำ:

- เริ่มจาก pretrained (`yolo8n.pt`)
- ใช้ `patience=50` (early stopping)
- Monitor mAP ทุก epoch
- Save checkpoint กลางทาง

✗ ไม่ทำ:

- เทรนจาก scratch (เปลืองเวลา)
- epochs > 500 (overfit)
- batch_size > RAM/GPU memory

16.3 Inference

✓ ทำ:

- ใช้ **cache** (image_hash + model_id + conf + iou)
- **Batch** ถ้ามีหลายรูป (≤ 32)
- **ONNX/TensorRT** สำหรับ production
- **Timeout** 30s

✗ ไม่ทำ:

- โหลด model ใหม่ทุก request
- Batch > 32 (OOM)
- รัน inference บน event loop (ใช้ `asyncio.to_thread`)

16.4 Security

✓ ทำ:

- เก็บ API key แบบ **encrypted** (aes-256)
- ห้าม **log** image binary / base64 / prompt
- **Validate** filename (path traversal)
- **Size limit** 20MB/รูป
- **RLS enforced** ทุก query

✗ ไม่ทำ:

- Hardcode credentials
- Query ข้าม tenant

- Return stacktrace ให้ client

16.5 Production Deployment

```
# 1. Build
pip install -r requirements.txt

# 2. Migrate DB
alembic upgrade head

# 3. Start
uvicorn app:app \
  --host 0.0.0.0 \
  --port 8000 \
  --workers 4 \
  --log-level info \
  --access-log

# 4. Behind Nginx (ตัวอย่าง)
```

Nginx config (ตัวอย่าง):

```
upstream yolo_app {
    server 127.0.0.1:8000;
    keepalive 32;
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate /etc/ssl/cert.pem;
    ssl_certificate_key /etc/ssl/key.pem;

    client_max_body_size 25M; # สำหรับ upload รูป

    location / {
        proxy_pass http://yolo_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 120s; # สำหรับ long-running inference
    }
}
```

17. ภาคผนวก

17.1 Module Types ที่รองรับ

Family	Size	RAM (train)	Speed	mAP (COCO)
YOLOv8n	Nano	2 GB	⚡ ⚡ ⚡ ⚡ ⚡	37.3
YOLOv8s	Small	4 GB	⚡ ⚡ ⚡ ⚡	44.9
YOLOv8m	Medium	8 GB	⚡ ⚡ ⚡	50.2
YOLOv8l	Large	12 GB	⚡ ⚡	52.9
YOLOv8x	XLarge	16 GB	⚡	53.9
YOLO11n	Nano	2 GB	⚡ ⚡ ⚡ ⚡ ⚡	39.5
YOLO11s	Small	4 GB	⚡ ⚡ ⚡ ⚡	47.0
YOLO11m	Medium	8 GB	⚡ ⚡ ⚡	51.5
YOLO11l	Large	12 GB	⚡ ⚡	53.4
YOLO11x	XLarge	16 GB	⚡	54.7

17.2 Error Codes

Code	HTTP	Meaning
DOMAIN_ERROR	400	Domain exception
DATASET_NOT_FOUND	404	Dataset not found
CLASS_NOT_FOUND	404	Class not found
IMAGE_NOT_FOUND	404	Image not found
ANNOTATION_NOT_FOUND	404	Annotation not found
TRAINING_NOT_FOUND	404	Training not found
MODEL_NOT_FOUND	404	Model not found
INFERENCE_NOT_FOUND	404	Inference not found

Code	HTTP	Meaning
INVALID_BBOX	422	BBox out of [0,1] range
INVALID_IMAGE	422	Corrupt / unsupported image
UNSUPPORTED_FORMAT	422	Format not supported
TRAINING_FAILED	500	Training pipeline failed
INFERENCE_FAILED	500	Inference failed
EXPORT_FAILED	500	ONNX/TensorRT export failed
GPU_UNAVAILABLE	503	No GPU available
RATE_LIMITED	429	Rate limit exceeded

17.3 Domain Events

Event	When
DatasetRegistered	หลัง create dataset
ImagesUploaded	หลัง upload รูป
AnnotationsCreated	หลัง create annotation
TrainingStarted	หลัง start training
TrainingCompleted	หลัง training สำเร็จ
ModelExported	หลัง export ONNX/TRT
InferenceServed	หลัง inference
ModelDriftDetected	เมื่อพบ drift

17.4 ตัวอย่าง

ย่อ	เต็ม
mAP	mean Average Precision
IoU	Intersection over Union

ย่อ	เต็ม
NMS	Non-Maximum Suppression
SSE	Server-Sent Events
RLS	Row-Level Security
bbox	Bounding Box
COCO	Common Objects in Context
TRT	TensorRT
LRU	Least Recently Used

17.5 Links

- [Ultralytics Docs](#)
- [YOLOv8 Paper](#)
- [YOLOv11](#)
- [COCO Metrics](#)
- [Albumentations](#)
- [ONNX Runtime](#)
- [TensorRT](#)
- [Roboflow](#)
- [Labellmg](#)

17.6 Quick Reference Card

```
# =====
# YOLO DETECTION – QUICK REFERENCE
# =====

# — Generate Module —
python create_module_yolo_detection.py all yolo 5 yolo --force

# — Migrate DB —
alembic upgrade head

# — Verify —
python create_module_yolo_detection.py verify yolo
python create_module_yolo_detection.py deps yolo
```

```
# — Run App —————
uvicorn app.app:app --reload --port 8000

# — Swagger —————
open http://localhost:8000/docs

# — Postman —————
# Import: docs/postman/yolo.json

# — Check Tables —————
psql $DATABASE_URL -c '\dt public.yolo_*'

# — Check RLS —————
psql $DATABASE_URL -c "SELECT tablename, rowsecurity FROM pg_tables
    WHERE schemaname='public' AND tablename LIKE 'yolo_%';"

# — Run Tests —————
pytest -m unit -v
pytest -m integration -v
pytest --cov=app.modules.yolo --cov-report=html
```

Support

- **Issues:** [GitHub Issues](#)
- **Docs:** docs/README_yolo.md
- **API Spec:** docs/API_yolo.md
- **OpenAPI:** <http://localhost:8000/docs>

✦ **Version:** 2.0.0 📅 **Last Update:** 2026-10-01 📄 **License:** MIT